Computer Science & IT

Compiler Design

(CD)

Introduction

Lecture 1



By- Ravindra Sir

Topics to be Covered



Topic

Topic

Topic

Intro

Introduction



Unstoppable Indians: Stories to Ignite Student Motivation

Anand Kumar – The Super 30 Hero A math genius from Bihar, Anand Kumar, couldn't study abroad due to financial issues. But instead of giving up, he decided to teach underprivileged students. He launched 'Super 30', coaching 30 poor kids for the IIT-JEE exam—for free. Most of his students cracked IITs! Anand turned rejection into revolution. If you think your problems are stopping you, think again. Sometimes, the biggest failures spark the brightest futures.



Unstoppable Indians: Stories to Ignite Student Motivation

Rani Gaidinliu – **The Forgotten Teen Freedom Fighter In the forests of Manipur, a 13-year-old girl stood against the British Empire. Her name? Rani Gaidinliu. She led a tribal movement, went to jail at 16, and spent 14 years behind bars—all for freedom. Most of us complain about homework. She fought for a nation at an age when we learn to ride a bicycle. Remember her courage next time you feel like giving up.**



Unstoppable Indians: Stories to Ignite Student Motivation

Kalpana Saroj – **From Child Bride to Business Tycoon Forced into marriage at 12. Abused. Nearly took her life. But Kalpana Saroj chose to rise. She ran away, worked in a garment factory, and slowly built her empire. Today, she's a Dalit multi-millionaire, running a company once worth 1000 crores. No degree. No privilege. Just unstoppable will. Failures don't define you—your comeback does.**



Unstoppable Indians: Stories to Ignite Student Motivation

Srinivasa Ramanujan – **The Self-Taught Math Genius He had no formal training, no fancy books—just a deep love for numbers. Srinivasa Ramanujan, a poor clerk from Tamil Nadu, filled notebooks with math the world had never seen. When British mathematicians finally read his work, they were stunned. He went on to revolutionize mathematics—all self-taught. No Wi-Fi. No coaching. Just passion. If you think talent needs tuition, remember Ramanujan. Genius isn't born in labs—it's born in quiet minds that don't give up.**

RBR

IISc, thesis - Compiler - S grade

c compiler → arr

Compiler design:

- Lexical analysis
- Parsing
- SDT
- Runtime environment
- Intermediate Code generation
- Local optimisation

Data flow analysis:

- Constant propogation
- Liveness analysis
- Common Subexpression elimination

(6 - 10) mins

Compiler design

→ The main of a compiler is to convert a high level language to low level language

→ John Bakus at IBM took 18 man years to introduce the first complete compiler for fortran in 1957. ✓

→

Building a Compiler requires knowledge of

→ Programming languages (parameter passing, scope of variables, memory allocation etc

→ Theory (automata, Context free grammars etc)

→ Algorithms and data structures (hash tables, graph algorithms, dynamic programming etc)

→ Computer architecture (assembly programming)

→ Software engineering

Interpreter :- An interpreter works similar to a compiler in most of the phases like lexical, Syntax and Semantic analysis. This analysis is performed for each single statement and when the statement is error free, instead of generating the corresponding code, that statement is executed and results of execution is displayed.

→ An interpreter every time the program has to be executed, it is processed and checked for the errors, it is slower than compiled programs

→ Some languages, like java are both Compiled and interpreted. The java program is first Compiled into byte code and the byte code is interpreted



Speed(compiled program) > (speed of Interpreted)

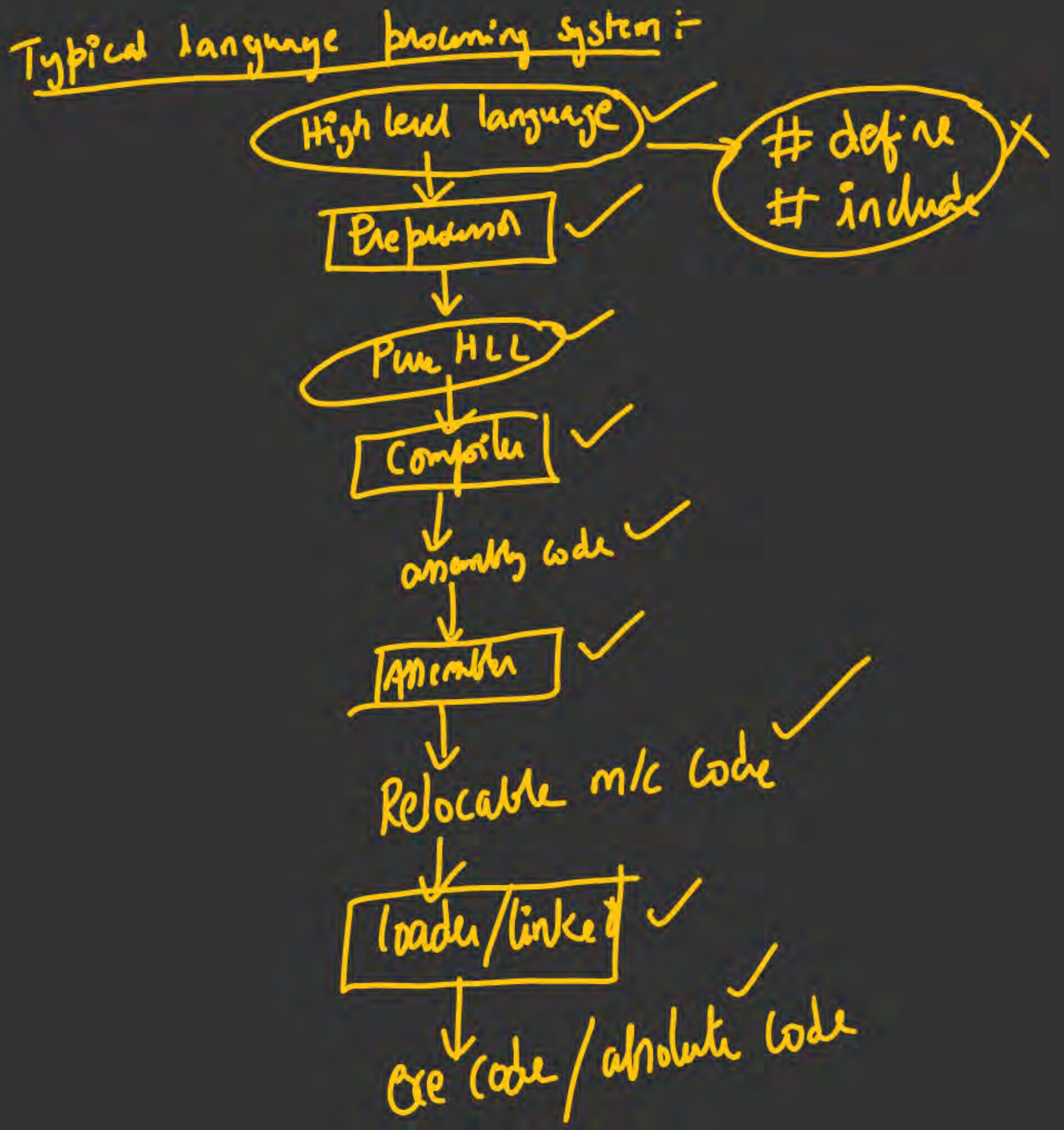
Adv of interpreter:

→ If a program is simple and it is executed once, an interpreter is preferred.

→ Interpreters are more portable than Compilers

→ Certain language features are supported by interpreters

Ex:- Consider dynamic types. In SNOBOL, the type of a variable is dependent on a value that is assigned to it anytime.


Typical language processing system :-
High level language
#define
#include
Preprocessor
Pure HLL
Compiler
assembly code
Assembler
Relocatable m/c code
Loader/linker
exe code / absolute code

Pre processor :-

a) macro expansion :- It performs macro expansion. All macros are identified and substituted with their respective values

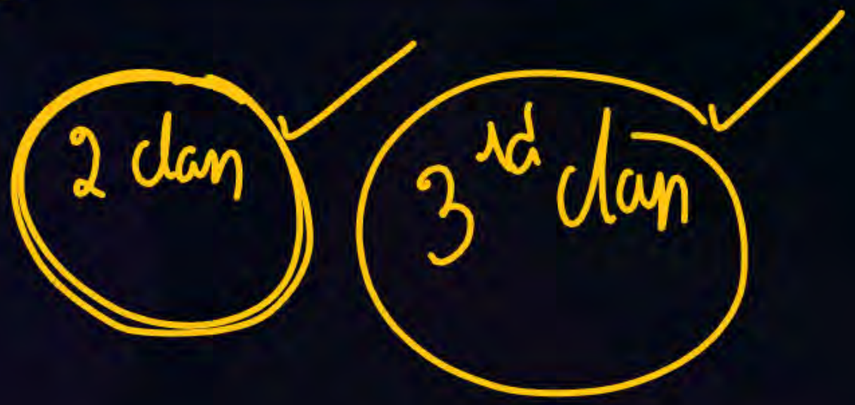
Ex :- #define Statements in C

#define Pie 3.14

Preprocessor

b) File inclusion : The Statements like #define <stdio.h> will be replaced by the files

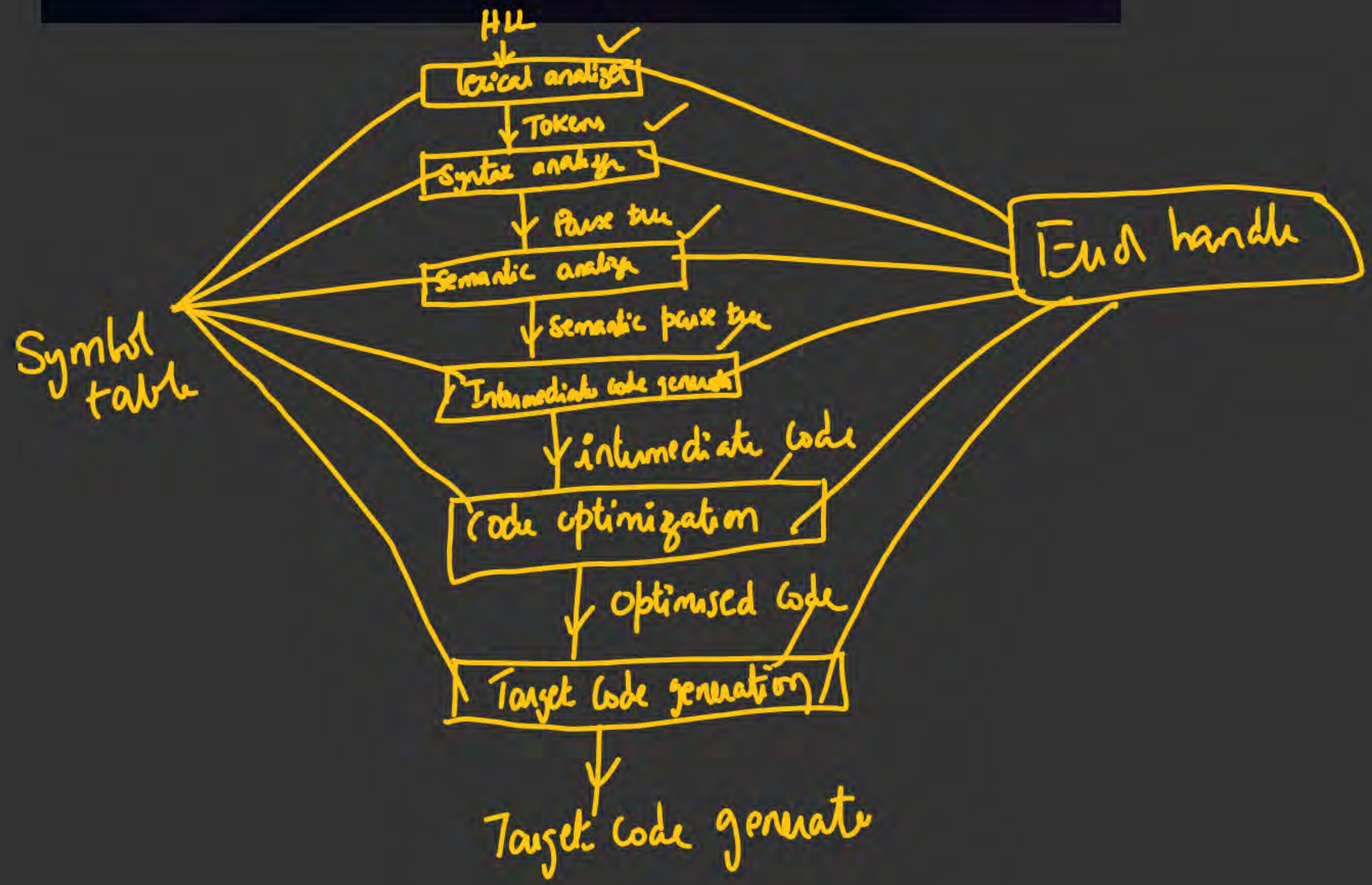
c) language expansion :- A preprocessor can add new functionalities to extend the language by built in macros. For example adding database query facilities to the language



Compiler :- Converts HLL into assembly code

Assembler :- Assembly language → m/c language

Loader/Linker :- OS


HL
Lexical analizer
Tokens
Syntax analyz
Parse tree
Semantic analiza
Semantic parse tree
Intermediate code generato
intermediate code
Code optimization
optimised code
Target code generation
Target code generato
Symbol table
Error handle

Design Phases of Compiler :-

Lexical analysis :-

It takes i/p stream of characters and convert into tokens

Token.

Keywords
identifiers
operators
Constants : numbers, characters, String

if (x >= y) x = 100

kw op id op id id op num

else y = 200;

kw id const

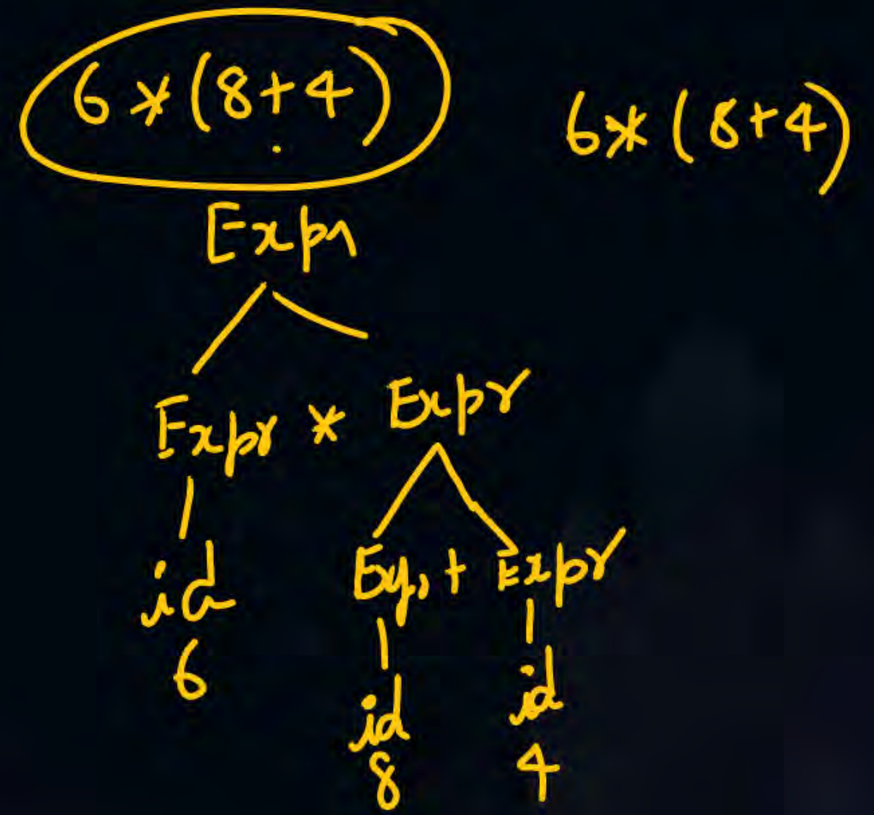
Syntax analyser

Syntax analyser checks if the syntax of a statement is correct

Expr → Expr + Expr
/ Expr * Expr
/ id
/ (Expr)

6 * (8 + 4)

6 * (8 + 4)



Semantic analysis :-

A + B

A → array

B → function

It verifies the meaning

Intermediate code generatn :-

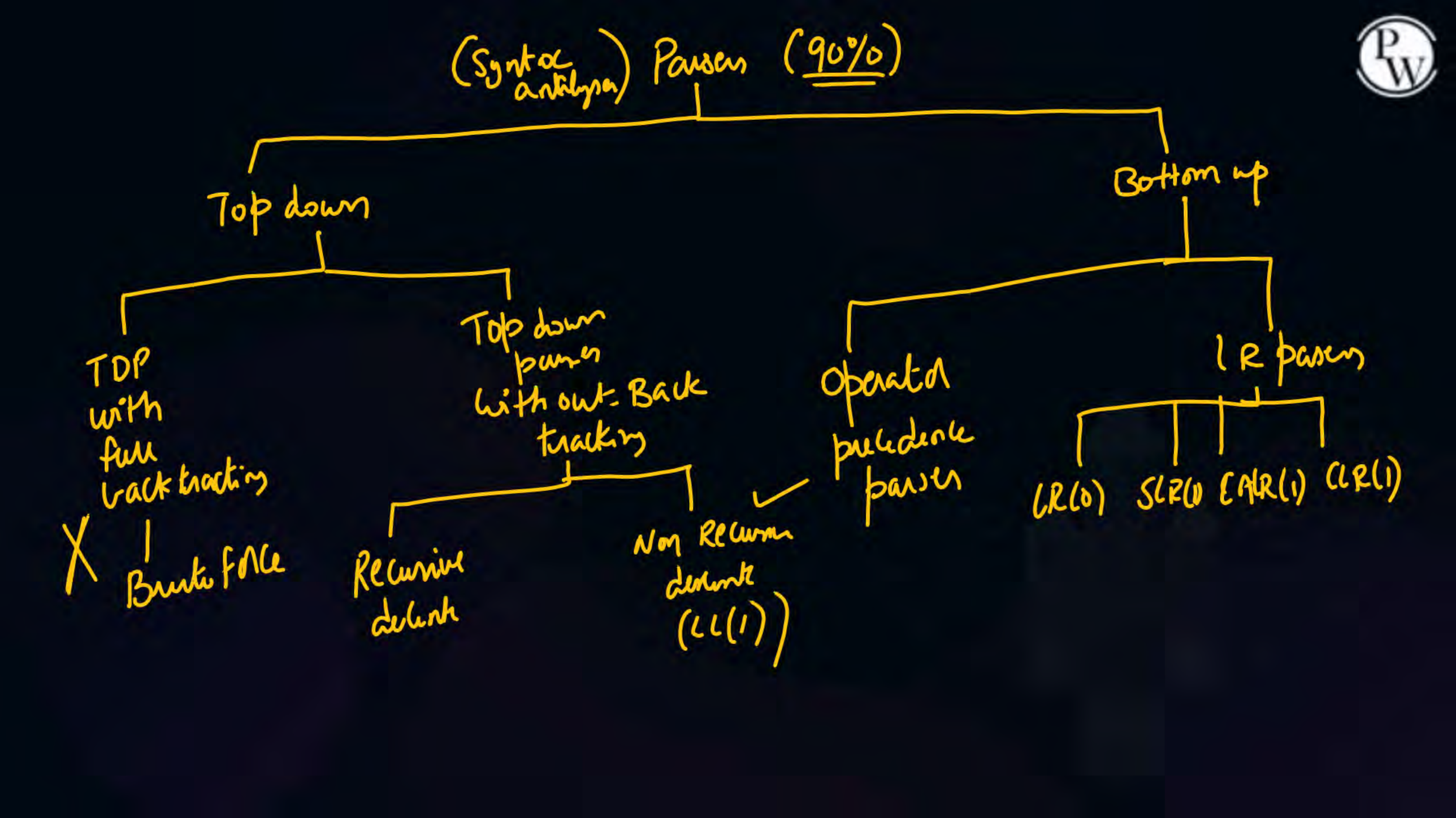
Intermediate code → generate

Code optimiser :-

Reduces no of line

Target code generator :-

It generates target code


(Syntax analysis) Parsers (90%)
Top down
Bottom up
TDP with full backtracking
X
Brute force
Top down parser without Back tracking
Recursive descent
Non Recursive descent (LL(1))
Operator precedence parser
LR parsers
LR(0)
SLR(1)
CLR(1)

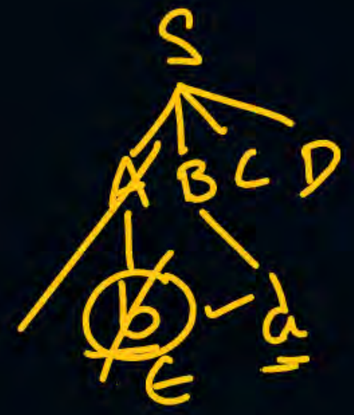
LL(1) parser

- Left to Right
- LMD
- lookahead

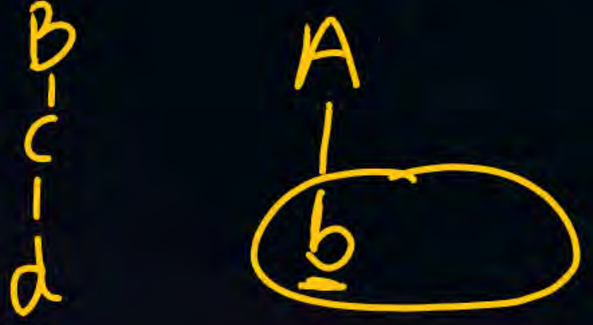
First(α)

follow(B)

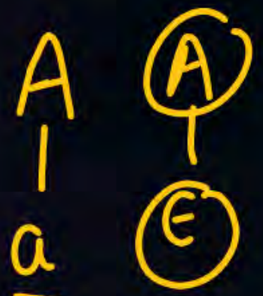
	First	follow
$S \to ABCD$	$\{b, d\}$	
$A \to b/\epsilon$	$\{b, \epsilon\}$	
$B \to c$	$\{c\}$	
$C \to d$	$\{d\}$	
$D \to e$	$\{e\}$	



First (α) gives set of all terminals that begin in a string derived by α

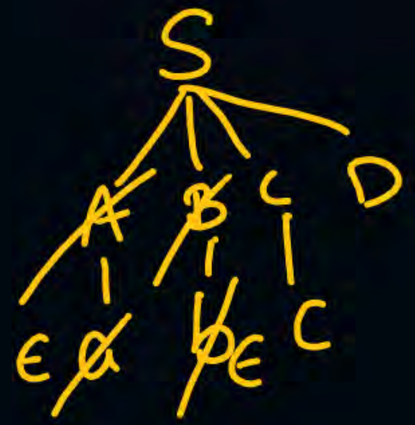


$S \to ABCD$	$\{a, b, c, d, \epsilon\}$
$A \to a/\epsilon$	$\{a, \epsilon\}$
$B \to b/\epsilon$	$\{b, \epsilon\}$
$C \to c/\epsilon$	$\{c, \epsilon\}$
$D \to d/\epsilon$	$\{d, \epsilon\}$





$S \to ABCD$	$\{a, b, c\}$
$A \to a/\epsilon$	$\{a, \epsilon\}$
$B \to b/\epsilon$	$\{b, \epsilon\}$
$C \to c.$	$\{c\}$
$D \to d/\epsilon$	$\{d, \epsilon\}$



$S \to Bb / Cd$	$\{a, b, c, d\}$
$B \to aB / \epsilon$	$\{a, \epsilon\}$
$C \to cC / \epsilon$	$\{c, \epsilon\}$



THANK - YOU